```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Mounted at /content/drive/
```

```
!pip install spectral
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting spectral
  Downloading spectral-0.23.1-py3-none-any.whl (212 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 212.9/212.9 KB 6.3 MB/s eta 0:00:00
Requirement already satisfied: numpy in /usr/local/lib/python3.8/dist-packages (from spectral) (1.22.4)
Installing collected packages: spectral
Successfully installed spectral-0.23.1
```

```
%tensorflow_version 2.x
import tensorflow as tf
import random
from tensorflow.keras import Sequential,layers
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import statistics
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, cohen_kappa_score
import tensorflow_probability as tfp
from operator import truediv
from tensorflow.compat.v1.distributions import Bernoulli
from plotly.offline import init_notebook_mode

import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import os
import spectral
import tensorflow as tf
from tensorflow.python.keras import backend as K

init_notebook_mode(connected=True)
%matplotlib inline
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
# Global variables
im_width, im_height, im_depth, im_channel = 11,11,30, 1
mc_loss_weight = 5
```

## Model Construction

```
def calc_euclidian_dists(x, y):
  # x : (n,d)
  # y : (m,d)
    n = x.shape[0]
    m = y.shape[0]
    x = tf.tile(tf.expand_dims(x, 1), [1, m, 1])
    y = tf.tile(tf.expand_dims(y, 0), [n, 1, 1])
    return tf.reduce_mean(tf.math.pow(x - y, 2), 2)
```

```
def _bernoulli(shape, mean):
    return tf.nn.relu(tf.sign(mean - tf.random.uniform(shape, minval=0, maxval=1, dtype=tf.float32)))
```

```
class DropBlock3D(tf.keras.layers.Layer):
    def __init__(self, keep_prob, block_size, scale=True, **kwargs):
        super(DropBlock3D, self).__init__(**kwargs)
        self.keep_prob = float(keep_prob) if isinstance(keep_prob, int) else keep_prob
        self.block_size = int(block_size)
        self.scale = tf.constant(scale, dtype=tf.bool) if isinstance(scale, bool) else scale

    def compute_output_shape(self, input_shape):
        return input_shape

    def build(self, input_shape):
        assert len(input_shape) == 5
        _, self.d, self.h, self.w, self.channel = input_shape.as_list()
```

```
        # pad the mask
        p0 = (self.block_size - 1) // 2
        p1 = (self.block_size - 1) - p0
        self.padding = [[0, 0], [p0, p1], [p0, p1], [p0, p1], [0, 0]]
        self.set_keep_prob()
        super(DropBlock3D, self).build(input_shape)

    def call(self, inputs, training=True, **kwargs):
        def drop():
            mask = self._create_mask(tf.shape(inputs))
            output = inputs * mask
            output = tf.cond(self.scale,
                             true_fn=lambda: output * tf.compat.v1.to_float(tf.size(mask)) / tf.reduce_sum(mask),
                             false_fn=lambda: output)
            return output

        if training is None:
            training = K.learning_phase()
        output = tf.cond(tf.logical_or(tf.logical_not(training), tf.equal(self.keep_prob, 1.0)),
                         true_fn=lambda: inputs,
                         false_fn=drop)
        return output

    def set_keep_prob(self, keep_prob=None):
        """This method only supports Eager Execution"""
        if keep_prob is not None:
            self.keep_prob = keep_prob
        d, w, h = tf.compat.v1.to_float(self.d), tf.compat.v1.to_float(self.w), tf.compat.v1.to_float(self.h)
        self.gamma = ((1. - self.keep_prob) * (d * w * h) / (self.block_size ** 3) /
                      ((d - self.block_size + 1) * (w - self.block_size + 1) * (h - self.block_size + 1)))

    def _create_mask(self, input_shape):
        sampling_mask_shape = tf.stack([input_shape[0],
                                        self.d - self.block_size + 1,
                                        self.h - self.block_size + 1,
                                        self.w - self.block_size + 1,
                                        self.channel])
        mask = _bernoulli(sampling_mask_shape, self.gamma)
        mask = tf.pad(mask, self.padding)
        mask = tf.nn.max_pool3d(mask, [1, self.block_size, self.block_size, self.block_size, 1], [1, 1, 1, 1, 1], 'SAME')
        mask = 1 - mask
        return mask
```

```
input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding='s
out1 = DropBlock3D(0.7,3)(out1)
out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
out2 = DropBlock3D(0.7,3)(out2)
out3 = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu')(out2)
out3 = layers.Reshape((out3.shape[1], out3.shape[2], out3.shape[3]*out3.shape[4]))(out3)
out3 = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(out3)
out4 = layers.Flatten()(out3)
out4 = layers.Dropout(0.4)(out4, training=True)
out4 = layers.Dense(256, activation='relu')(out4)
out5 = layers.Dropout(0.4)(out4,training=True)
out5 = layers.Dense(128, activation='relu')(out5)
#out5 = layers.Dropout(0.4)(out5, training=True)
#out6 = layers.Dense(16, activation='relu')(out5)
model = Model(inputs=input_layer,outputs=out5)
model.summary()
```

```
WARNING:tensorflow:From /usr/local/lib/python3.8/dist-packages/tensorflow/python/util/dispatch.py:1176: to_float (from tensorflow.p
Instructions for updating:
Use `tf.cast` instead.
WARNING:tensorflow:From /usr/local/lib/python3.8/dist-packages/tensorflow/python/autograph/pyct/static_analysis/liveness.py:83: Ana
Instructions for updating:
Lambda fuctions will be no more assumed to be used in the statement where they are used, or at least in the same block. https://git
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 11, 11, 30, 1)]   0

 conv3d (Conv3D)             (None, 11, 11, 30, 8)     512

 drop_block3d (DropBlock3D)  (None, 11, 11, 30, 8)     0

 conv3d_1 (Conv3D)           (None, 11, 11, 30, 16)    5776

 drop_block3d_1 (DropBlock3D  (None, 11, 11, 30, 16)   0
 )

 conv3d_2 (Conv3D)           (None, 9, 9, 28, 32)      13856
```

```
 reshape (Reshape)           (None, 9, 9, 896)         0

 conv2d (Conv2D)             (None, 7, 7, 64)          516160

 flatten (Flatten)           (None, 3136)              0

 dropout (Dropout)           (None, 3136)              0

 dense (Dense)               (None, 256)               803072

 dropout_1 (Dropout)         (None, 256)               0

 dense_1 (Dense)             (None, 128)               32896

=================================================================
Total params: 1,372,272
Trainable params: 1,372,272
Non-trainable params: 0
_________________________________________________________________
```

```
class Prototypical(Model):
    def __init__(self, model, w, h, d, c):
        super(Prototypical, self).__init__()
        self.w, self.h, self.d, self.c = w, h, d, c
        self.encoder = model

    def call(self, support, query, support_labels, query_labels, K, C, N,n_times,training=True):
      n_class = C                                                          #10
      n_support = K                                                        #5
      n_query = N                                                          #15

      if training == True :
        loss = 0
        mc_predictions = []                                                # list of predictions for multiple passes
        for i in range(n_times) :
          y = np.zeros((int(C*N),C))                                         #(150,10)
          for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                      # creation of 1-hot for true labels
            y[i][x] = 1.                                             # n_times passing every query image for calculating variance
          cat = tf.concat([support,query], axis=0)                         # [200,9,9,20,1])
          z = self.encoder(cat)                                            # [200, 320]
          # Divide embedding into support and query
          z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
          # Prototypes are means of n_support examples
          z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)         #[10, 320]
          z_query = z[n_class * n_support:]                                #[150, 320]
          # Calculate distances between query and prototypes
          dists = calc_euclidian_dists(z_query, z_prototypes)              #[150, 10]
          # log softmax of calculated distances
          log_p_y = tf.nn.log_softmax(-dists, axis=-1)                     #[150, 10]
          loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))   #loss for the current pass
          loss += loss1                                                    # adding loss for each pass
          predictions = tf.nn.softmax(-dists, axis=-1)                            # prediction probability for the search-space clas
          mc_predictions.append(predictions)

        y = np.zeros((int(C*N),C))
        for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                      # creation of 1-hot for true labels
            y[i][x] = 1.
        mc_predictions = tf.convert_to_tensor(np.reshape(np.asarray(mc_predictions),(n_times,int(C*N),C)))  #(n_times,150,10)
        std_predictions = tf.math.reduce_std(mc_predictions,axis=0)
        std = tf.reduce_sum(tf.reduce_sum(tf.multiply(std_predictions,y),axis=1))
        # print('std', std)
        loss += mc_loss_weight*std
        temp_list.append(std)
        # calculating mean accuracy
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)         # mean prediction probability for each class (150,10)
        mean_eq = tf.cast(tf.equal(                                      # accuracy for the current pass
            tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
        mean_accuracy = tf.reduce_mean(mean_eq)
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)         # mean prediction probability for each class (150,10)
        return loss, mean_accuracy, mean_predictions

      if training == False :
        loss = 0
        mc_predictions = []                                              # list of predictions for multiple passes
        for i in range(n_times) :                                        # n_times passing the query images for variance calculati
          y = np.zeros((int(C*N),C))                                     # (150,10)
          for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                    # creation of 1-hot for the true labels
            y[i][x] = 1.
```

```
        # merge support and query to forward through encoder
        cat = tf.concat([support,query], axis=0)                          # [200,9,9,20,1]
        z = self.encoder(cat)                                             # [200, 320]
        # Divide embedding into support and query
        z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
        # Prototypes are means of n_support examples
        z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)          #[10, 320]
        z_query = z[n_class * n_support:]                                 #[150, 320]
        # Calculate distances between query and prototypes
        dists = calc_euclidian_dists(z_query, z_prototypes)               #[150, 10]
        # log softmax of calculated distances
        log_p_y = tf.nn.log_softmax(-dists, axis=-1)                      #[150, 10]
        loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))
        loss += loss1
        predictions = tf.nn.softmax(-dists, axis=-1)                              # prediction probabilities for the classes for cur
        mc_predictions.append(predictions)
      y = np.zeros((int(C*N),C))                                        # (150,10)
      for i in range(int(C*N)) :
          x = support_labels.index(query_labels[i])                     # creation of 1-hot for the true labels
          y[i][x] = 1.
      mean_predictions = tf.reduce_mean(mc_predictions,axis=0)          # mean prediction probability for each class (150,10)
      mean_eq = tf.cast(tf.equal(                                       # accuracy for the current pass
          tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
          tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
      mean_accuracy = tf.reduce_mean(mean_eq)
      mean_pred_index = tf.argmax(mean_predictions,axis=1)
      # mean class-wise accuracies
      mean_correct_class = [[] for i in range(tC)]
      mean_correct_pred = [[] for i in range(tC)]
      classwise_mean_acc = [[] for i in range(tC)]
      for i in range(int(C*N)):
        x = support_labels.index(query_labels[i])
        mean_correct_class[x].append('4')
        if(mean_pred_index[i] == x) :
          mean_correct_pred[x].append('4')
      for i in range(tC) :
         z = len(mean_correct_pred[i])/len(mean_correct_class[i])
         classwise_mean_acc[i].append(z)
      #std calculation
      std = 0
      for i in range(int(C*N)) :
         x = support_labels.index(query_labels[i])
         p_i = np.array([p[i,:] for p in mc_predictions])
         std_i = tf.math.reduce_std(p_i,axis=0)
         std_i_true = std_i[x]
         std += std_i_true                                              # adding std of each class
      # print('std',std)
      loss += mc_loss_weight*std
      y = np.zeros((int(C*N),C))                                        # (150,10)
      for i in range(int(C*N)) :
          x = support_labels.index(query_labels[i])                     # creation of 1-hot for the true labels
          y[i][x] = 1.
      return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y


    def save(self, model_path):
      self.encoder.save(model_path)

    def load(self, model_path):
      self.encoder(tf.zeros([1, self.w, self.h, self.c]))
      self.encoder.load_weights(model_path)
```

## Data Loading

```
def loadData(name):
    data = sio.loadmat('/content/drive/My Drive/Data/indian_pines_corrected.mat')['indian_pines_corrected']
    labels = sio.loadmat('/content/drive/My Drive/Data/indian_pines_gt.mat')['indian_pines_gt']
    return data, labels
# without reduction of 200 channels to 30 channels, memory error while creating cube
def applyPCA(X, numComponents):
    newX = np.reshape(X, (-1, X.shape[2]))
    pca = PCA(n_components=numComponents, whiten=True)
    newX = pca.fit_transform(newX)
    newX = np.reshape(newX, (X.shape[0],X.shape[1], numComponents))
    return newX, pca

def padWithZeros(X, margin):
    newX = np.zeros((X.shape[0] + 2 * margin, X.shape[1] + 2* margin, X.shape[2]))
    x_offset = margin
    y_offset = margin
```

```
    newX[x_offset:X.shape[0] + x_offset, y_offset:X.shape[1] + y_offset, :] = X
    return newX

def createImageCubes(X, y, windowSize, removeZeroLabels = True):
    margin = int((windowSize - 1) / 2)
    zeroPaddedX = padWithZeros(X, margin=margin)  # X :(145, 145, 30) --> (195, 195, 30) with window =25
    # split patches
    patchesData = np.zeros((X.shape[0] * X.shape[1], windowSize, windowSize, X.shape[2]))  # (21025, 25, 25, 30)
    patchesLabels = np.zeros((X.shape[0] * X.shape[1]))  # (21025,)
    patchIndex = 0

    for r in range(margin, zeroPaddedX.shape[0] - margin):
        for c in range(margin, zeroPaddedX.shape[1] - margin):
            patch = zeroPaddedX[r - margin:r + margin + 1, c - margin:c + margin + 1]
            patchesData[patchIndex, :, :, :] = patch
            patchesLabels[patchIndex] = y[r-margin, c-margin]
            patchIndex = patchIndex + 1

    patchesData = np.expand_dims(patchesData, axis=-1)
    return patchesData,patchesLabels


dataset1 = 'indian_pines'                                        # 16 classes
ip_x1, ip_y = loadData(dataset1)                              #((512, 217, 204), (512, 217))
ip_x2,pca = applyPCA(ip_x1,numComponents=30)                   # ((512, 217, 20), (512, 217))
ip_X,ip_Y = createImageCubes(ip_x2, ip_y, windowSize=11)   #(111104, 9, 9, 20, 1) (111104,)
print(ip_X.shape,ip_Y.shape)
```

```
(21025, 11, 11, 30, 1) (21025,)
```

```
def patches_class(X,Y,n) :
  n_classes = n
  patches_list = []
  for i in range(1,n_classes+1):   # not considering class 0
    patchesData_Ith_Label = X[Y==i,:,:,:,:]
    patches_list.append(patchesData_Ith_Label)
  return patches_list


patches_class_ip = patches_class(ip_X,ip_Y,16) # class_wise list of patches #(16,) for class 0: (2009, 9, 9, 20, 1)


train_class_indices = [1,2,4,5,7,9,10,11,13,14]
test_class_indices = [0,3,6,8,12,15]
train_patches_class = [patches_class_ip[i] for i in train_class_indices]        #(10)
test_patches_class = [patches_class_ip[i] for i in test_class_indices]        #(6)
train_class_labels = [2,3,5,6,8,10,11,12,14,15]
test_class_labels = [1,4,7,9,13,16]     #[11...16]
```

### Prepare Training and validation Dataset

### Create DataLoader

```
C = 5  # n_class
K1 = 5   # n_support
N = 15   # n_query
tC = 3   # classes in a test episode
im_height,im_width,im_depth = 11,11,30


def new_episode(patches_list,K,C,N,class_labels) :
  selected_classes = np.random.choice(class_labels,C,replace=False)  # Randomly choice 5 Classes out of classes available
  tsupport_patches = []
  tquery_patches = []
  query_labels = []
  support_labels = list(selected_classes)
  for x in selected_classes :
    sran_indices = np.random.choice(len(patches_list[x-1]),K,replace=False)  # for class no X-1: select random sample no
    support_patches = patches_list[x-1][sran_indices,:,:,:,:]
    qran_indices = np.random.choice(len(patches_list[x-1]),N,replace=False)  # N Samples for Query
    query_patches = patches_list[x-1][qran_indices,:,:,:,:]
  # Support and Query patches belong to same Class
    for i in range(N) :
      query_labels.append(x)    # N Samples for Query
    tquery_patches.extend(query_patches)
    tsupport_patches.extend(support_patches)
  temp1 = list(zip(tquery_patches, query_labels))
  random.shuffle(temp1)        # By Doing Shuffling, Support, Query Same class combination got mismatched - mitigated by support index
  tquery_patches, query_labels = zip(*temp1)
  tquery_patches = tf.convert_to_tensor(np.reshape(np.asarray(tquery_patches),(C*N,im_height,im_width,im_depth,1)),dtype=tf.float32)
```

```
  tsupport_patches = tf.convert_to_tensor(np.reshape(np.asarray(tsupport_patches),(C*K,im_height,im_width,im_depth,1)),dtype=tf.float32)
  return tquery_patches, tsupport_patches, query_labels, support_labels
```

```
tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
```

## Training

```
ProtoModel = Prototypical(model,im_width, im_height, im_depth, im_channel)
optimizer = tf.keras.optimizers.Adam(0.00001)          #Adam(0.001)
n_times = 15
#std_list = []
```

```
# Metrics to gather
train_loss = tf.metrics.Mean(name='train_loss')
train_acc = tf.metrics.Mean(name='train_accuracy')

def train_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
    train_loss(loss)
    train_acc(mean_accuracy)
```

```
checkpoint_dir = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP/
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
for epoch in range(50): # n_epochs-140
    train_loss.reset_states()
    train_acc.reset_states()

    for epi in range(n_episodes):
        tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
        train_step(tsupport_patches, tquery_patches,support_labels, query_labels, K1, C, N)
        template = 'Epoch {}, Episode {}, Train Loss: {:.2f}, Train Accuracy: {:.2f}'
        print(template.format(epoch+1, epi+1,train_loss.result(),train_acc.result()*100))
    if epoch % 5 == 0 and epoch != 0 :
        checkpoint.save(file_prefix = checkpoint_prefix)
```

```
std tf.Tensor(0.007663389, shape=(), dtype=float32)
WARNING:tensorflow:5 out of the last 5 calls to <function _BaseOptimizer._update_step_xla at 0x7f09b948
WARNING:tensorflow:6 out of the last 6 calls to <function _BaseOptimizer._update_step_xla at 0x7f09b948
```

**Streaming output truncated to the last 5000 lines.**

```
Epoch 16, Episode 11, Train Loss: 40.08, Train Accuracy: 71.27
std tf.Tensor(8.692383, shape=(), dtype=float32)
Epoch 16, Episode 12, Train Loss: 40.82, Train Accuracy: 73.22
std tf.Tensor(5.9748836, shape=(), dtype=float32)
Epoch 16, Episode 13, Train Loss: 40.55, Train Accuracy: 74.05
std tf.Tensor(7.122345, shape=(), dtype=float32)
Epoch 16, Episode 14, Train Loss: 40.74, Train Accuracy: 74.29
std tf.Tensor(10.0161495, shape=(), dtype=float32)
Epoch 16, Episode 15, Train Loss: 41.80, Train Accuracy: 75.11
std tf.Tensor(5.4707294, shape=(), dtype=float32)
Epoch 16, Episode 16, Train Loss: 41.72, Train Accuracy: 75.00
std tf.Tensor(6.2752666, shape=(), dtype=float32)
Epoch 16, Episode 17, Train Loss: 41.42, Train Accuracy: 76.16
std tf.Tensor(5.2734785, shape=(), dtype=float32)
Epoch 16, Episode 18, Train Loss: 40.87, Train Accuracy: 76.89
std tf.Tensor(5.4785233, shape=(), dtype=float32)
Epoch 16, Episode 19, Train Loss: 40.63, Train Accuracy: 76.77
std tf.Tensor(5.1464114, shape=(), dtype=float32)
Epoch 16, Episode 20, Train Loss: 40.47, Train Accuracy: 75.93
std tf.Tensor(6.8644342, shape=(), dtype=float32)
Epoch 16, Episode 21, Train Loss: 40.67, Train Accuracy: 75.87
std tf.Tensor(7.0096827, shape=(), dtype=float32)
Epoch 16, Episode 22, Train Loss: 40.66, Train Accuracy: 76.91
std tf.Tensor(5.8991065, shape=(), dtype=float32)
Epoch 16, Episode 23, Train Loss: 40.58, Train Accuracy: 76.87
std tf.Tensor(4.9722233, shape=(), dtype=float32)
Epoch 16, Episode 24, Train Loss: 40.22, Train Accuracy: 77.17
std tf.Tensor(7.2247024, shape=(), dtype=float32)
Epoch 16, Episode 25, Train Loss: 40.40, Train Accuracy: 77.44
std tf.Tensor(6.453219, shape=(), dtype=float32)
Epoch 16, Episode 26, Train Loss: 40.40, Train Accuracy: 77.44
std tf.Tensor(4.5788255, shape=(), dtype=float32)
Epoch 16, Episode 27, Train Loss: 40.19, Train Accuracy: 77.04
std tf.Tensor(9.3407955, shape=(), dtype=float32)
Epoch 16, Episode 28, Train Loss: 40.82, Train Accuracy: 77.05
std tf.Tensor(5.612718, shape=(), dtype=float32)
Epoch 16, Episode 29, Train Loss: 40.51, Train Accuracy: 77.61
std tf.Tensor(8.378391, shape=(), dtype=float32)
Epoch 16, Episode 30, Train Loss: 40.76, Train Accuracy: 77.96
std tf.Tensor(6.729659, shape=(), dtype=float32)
Epoch 16, Episode 31, Train Loss: 40.67, Train Accuracy: 78.41
std tf.Tensor(8.199577, shape=(), dtype=float32)
Epoch 16, Episode 32, Train Loss: 41.01, Train Accuracy: 78.17
std tf.Tensor(6.757858, shape=(), dtype=float32)
Epoch 16, Episode 33, Train Loss: 41.04, Train Accuracy: 78.46
std tf.Tensor(9.224477, shape=(), dtype=float32)
Epoch 16, Episode 34, Train Loss: 41.41, Train Accuracy: 78.71
std tf.Tensor(5.117579, shape=(), dtype=float32)
Epoch 16, Episode 35, Train Loss: 41.36, Train Accuracy: 78.36
std tf.Tensor(4.6693034, shape=(), dtype=float32)
Epoch 16, Episode 36, Train Loss: 41.05, Train Accuracy: 78.52
std tf.Tensor(9.397895, shape=(), dtype=float32)
Epoch 16, Episode 37, Train Loss: 41.36, Train Accuracy: 78.77
std tf.Tensor(8.394308, shape=(), dtype=float32)
Epoch 16, Episode 38, Train Loss: 41.60, Train Accuracy: 78.84
std tf.Tensor(8.058957, shape=(), dtype=float32)
Epoch 16, Episode 39, Train Loss: 41.71, Train Accuracy: 79.21
std tf.Tensor(4.4557858, shape=(), dtype=float32)
Epoch 16, Episode 40, Train Loss: 41.50, Train Accuracy: 78.77
std tf.Tensor(6.6406302, shape=(), dtype=float32)
Epoch 16, Episode 41, Train Loss: 41.42, Train Accuracy: 79.09
std tf.Tensor(9.2817, shape=(), dtype=float32)
Epoch 16, Episode 42, Train Loss: 41.71, Train Accuracy: 79.14
std tf.Tensor(6.8592606, shape=(), dtype=float32)
Epoch 16, Episode 43, Train Loss: 41.61, Train Accuracy: 79.44
std tf.Tensor(4.3179135, shape=(), dtype=float32)
Epoch 16, Episode 44, Train Loss: 41.51, Train Accuracy: 79.00
std tf.Tensor(6.442285, shape=(), dtype=float32)
Epoch 16, Episode 45, Train Loss: 41.40, Train Accuracy: 79.38
std tf.Tensor(9.105317, shape=(), dtype=float32)
Epoch 16, Episode 46, Train Loss: 41.74, Train Accuracy: 79.28
std tf.Tensor(7.102624, shape=(), dtype=float32)
Epoch 16, Episode 47, Train Loss: 41.77, Train Accuracy: 79.29
std tf.Tensor(6.179592, shape=(), dtype=float32)
Epoch 16, Episode 48, Train Loss: 41.61, Train Accuracy: 79.58
std tf.Tensor(7.7979913, shape=(), dtype=float32)
Epoch 16, Episode 49, Train Loss: 41.71, Train Accuracy: 79.65
std tf.Tensor(8.946354, shape=(), dtype=float32)
Epoch 16, Episode 50, Train Loss: 41.92, Train Accuracy: 79.81
std tf.Tensor(3.145298, shape=(), dtype=float32)
Epoch 16, Episode 51, Train Loss: 41.60, Train Accuracy: 79.53
std tf.Tensor(5.7616243, shape=(), dtype=float32)
Epoch 16, Episode 52, Train Loss: 41.49, Train Accuracy: 79.67
std tf.Tensor(7.6351748, shape=(), dtype=float32)
Epoch 16, Episode 53, Train Loss: 41.48, Train Accuracy: 80.00
std tf.Tensor(7.5777993, shape=(), dtype=float32)
```

```
Epoch 16, Episode 54, Train Loss: 41.55, Train Accuracy: 80.10
```

### Tuning_5

```
std tf.Tensor(8.898748, shape=(), dtype=float32)
```

```
tune_set_5 = [[] for i in range(6)]
for j in range(6) :
  tune_set_5[j] = test_patches_class[j][:5,:,:,:,:]   # for each class first 5 samples taken
std_tune_5 = []
```

```
std tf.Tensor(4.698825, shape=(), dtype=float32)
```

```
def tune_episode(tune_set,tC,tK,tN,test_class_labels) :
  selected_classes = np.random.choice(test_class_labels,tC,replace=False)
  support_labels  = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    np.random.shuffle(tune_set[y])
    support_imgs = tune_set[y][:tK,:,:,:,:]    #Support 1, Query 4
    query_imgs = tune_set[y][tK:5,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(tN) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(tC*tN,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(tC*tK,im_height,im_width,im_depth,1)),dtype=tf.float32)
  return query_patches, support_patches, query_labels, support_labels
```

```
Epoch 16, Episode 71, Train Loss: 41.72, Train Accuracy: 80.08
```

```
query_patches, support_patches, query_labels, support_labels = tune_episode(tune_set_5,4,1,4,test_class_labels)
```

```
std tf.Tensor(8.362537, shape=(), dtype=float32)
```

```
# Metrics to gather
tune_loss = tf.metrics.Mean(name='tune_loss')
tune_acc = tf.metrics.Mean(name='tune_accuracy')

def tune_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
    tune_loss(loss)
    tune_acc(mean_accuracy)
```

```
Epoch 16, Episode 81, Train Loss: 41.79, Train Accuracy: 79.95
```

```
checkpoint_dir1 = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IF
checkpoint_prefix1 = os.path.join(checkpoint_dir1, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
std_tune_5 = []
for epoch in range(41):
    tune_loss.reset_states()
    tune_acc.reset_states()
    for epi in range(n_episodes+1):
        tquery_patches, tsupport_patches, query_labels, support_labels = tune_episode(tune_set_5,3,1,4,test_class_labels)
        tune_step(tsupport_patches, tquery_patches,support_labels, query_labels, 1, 3, 4)

    template = 'Epoch {}, Tune Loss: {:.2f}, Tune Accuracy: {:.2f}'
    print(template.format(epoch+1,tune_loss.result(),tune_acc.result()*100))
    if (epoch+1)%5 == 0 :
      checkpoint.save(file_prefix = checkpoint_prefix1)
```

```
std tf.Tensor(2.5119426, shape=(), dtype=float32)
std tf.Tensor(2.6468234, shape=(), dtype=float32)
std tf.Tensor(2.2208383, shape=(), dtype=float32)
std tf.Tensor(2.776962, shape=(), dtype=float32)
std tf.Tensor(1.5350839, shape=(), dtype=float32)
std tf.Tensor(1.2779937, shape=(), dtype=float32)
std tf.Tensor(1.1183547, shape=(), dtype=float32)
std tf.Tensor(2.7457695, shape=(), dtype=float32)
std tf.Tensor(0.8717665, shape=(), dtype=float32)
std tf.Tensor(2.6091466, shape=(), dtype=float32)
std tf.Tensor(2.3066912, shape=(), dtype=float32)
std tf.Tensor(2.094003, shape=(), dtype=float32)
std tf.Tensor(1.4011083, shape=(), dtype=float32)
std tf.Tensor(1.838781, shape=(), dtype=float32)
std tf.Tensor(2.0805821, shape=(), dtype=float32)
std tf.Tensor(1.248991, shape=(), dtype=float32)
std tf.Tensor(2.379145, shape=(), dtype=float32)
std tf.Tensor(1.34669, shape=(), dtype=float32)
std tf.Tensor(1.5788496, shape=(), dtype=float32)
std tf.Tensor(2.077191, shape=(), dtype=float32)
std tf.Tensor(2.0242434, shape=(), dtype=float32)
std tf.Tensor(1.0528151, shape=(), dtype=float32)
std tf.Tensor(1.1246116, shape=(), dtype=float32)
std tf.Tensor(1.8391815, shape=(), dtype=float32)
std tf.Tensor(1.6245056, shape=(), dtype=float32)
std tf.Tensor(1.3307323, shape=(), dtype=float32)
std tf.Tensor(1.3878989, shape=(), dtype=float32)
std tf.Tensor(1.7447915, shape=(), dtype=float32)
std tf.Tensor(1.2123778, shape=(), dtype=float32)
std tf.Tensor(1.3756893, shape=(), dtype=float32)
std tf.Tensor(1.1436882, shape=(), dtype=float32)
std tf.Tensor(1.4917197, shape=(), dtype=float32)
std tf.Tensor(0.60925364, shape=(), dtype=float32)
std tf.Tensor(1.0302696, shape=(), dtype=float32)
std tf.Tensor(0.2658014, shape=(), dtype=float32)
std tf.Tensor(1.3086094, shape=(), dtype=float32)
std tf.Tensor(1.9312932, shape=(), dtype=float32)
std tf.Tensor(0.57862014, shape=(), dtype=float32)
std tf.Tensor(0.93621296, shape=(), dtype=float32)
std tf.Tensor(1.0830567, shape=(), dtype=float32)
std tf.Tensor(0.90375566, shape=(), dtype=float32)
std tf.Tensor(0.8449109, shape=(), dtype=float32)
std tf.Tensor(0.7845855, shape=(), dtype=float32)
std tf.Tensor(0.57713, shape=(), dtype=float32)
std tf.Tensor(1.0958586, shape=(), dtype=float32)
std tf.Tensor(0.2936899, shape=(), dtype=float32)
std tf.Tensor(1.4820724, shape=(), dtype=float32)
std tf.Tensor(1.060088, shape=(), dtype=float32)
std tf.Tensor(0.6082795, shape=(), dtype=float32)
std tf.Tensor(0.6200352, shape=(), dtype=float32)
std tf.Tensor(0.08262595, shape=(), dtype=float32)
std tf.Tensor(1.266478, shape=(), dtype=float32)
std tf.Tensor(0.88436955, shape=(), dtype=float32)
std tf.Tensor(0.06041939, shape=(), dtype=float32)
std tf.Tensor(0.4776972, shape=(), dtype=float32)
std tf.Tensor(0.8249241, shape=(), dtype=float32)
std tf.Tensor(0.45086268, shape=(), dtype=float32)
std tf.Tensor(0.9068106, shape=(), dtype=float32)
std tf.Tensor(1.6432287, shape=(), dtype=float32)
std tf.Tensor(0.46008462, shape=(), dtype=float32)
std tf.Tensor(1.1916833, shape=(), dtype=float32)
std tf.Tensor(0.3630861, shape=(), dtype=float32)
std tf.Tensor(0.61453533, shape=(), dtype=float32)
std tf.Tensor(0.76510584, shape=(), dtype=float32)
std tf.Tensor(1.4923291, shape=(), dtype=float32)
std tf.Tensor(0.56437325, shape=(), dtype=float32)
std tf.Tensor(1.4593391, shape=(), dtype=float32)
std tf.Tensor(0.6312758, shape=(), dtype=float32)
std tf.Tensor(0.026595602, shape=(), dtype=float32)
std tf.Tensor(0.11948984, shape=(), dtype=float32)
std tf.Tensor(0.17654014, shape=(), dtype=float32)
std tf.Tensor(0.4440673, shape=(), dtype=float32)
std tf.Tensor(1.0395143, shape=(), dtype=float32)
std tf.Tensor(0.66199905, shape=(), dtype=float32)
std tf.Tensor(0.42596143, shape=(), dtype=float32)
std tf.Tensor(0.39652628, shape=(), dtype=float32)
std tf.Tensor(0.61683506, shape=(), dtype=float32)
std tf.Tensor(1.1331568, shape=(), dtype=float32)
std tf.Tensor(0.892488, shape=(), dtype=float32)
std tf.Tensor(0.7981694, shape=(), dtype=float32)
std tf.Tensor(0.914605, shape=(), dtype=float32)
std tf.Tensor(1.0937932, shape=(), dtype=float32)
std tf.Tensor(0.3382666, shape=(), dtype=float32)
std tf.Tensor(1.2008175, shape=(), dtype=float32)
std tf.Tensor(0.7350445, shape=(), dtype=float32)
std tf.Tensor(1.3952191, shape=(), dtype=float32)
std tf.Tensor(0.82837456, shape=(), dtype=float32)
std tf.Tensor(1.0013242, shape=(), dtype=float32)
std tf.Tensor(1.0067501, shape=(), dtype=float32)
std tf.Tensor(0.5778327, shape=(), dtype=float32)
```

```
std tf.Tensor(0.8706957, shape=(), dtype=float32)
```

```
def test_episode(test_patches_class,test_class_labels,test_C,test_K,i,f) :
  selected_classes = test_class_labels[i:f]   # [1, 2, 3, 4, 5, 6, 7, 8]
  support_labels = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    support_imgs = test_patches_class[y][:test_K,:,:,:,:]
    query_imgs = test_patches_class[y][test_K:,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(query_imgs.shape[0]) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  x = len(query_labels)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(x,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(test_C*test_K,im_height,im_width,im_depth,1)),dtype=tf.f
  return query_patches, support_patches, query_labels, support_labels,x
```

```
std tf.Tensor(0.34567893, shape=(), dtype=float32)
```

```
query_patches, support_patches, query_labels, support_labels,x = test_episode(test_patches_class,test_class_labels,3,5,0,3)
```

```
std tf.Tensor(0.41137272, shape=(), dtype=float32)
```

```
# Metrics to gather
test_loss = tf.metrics.Mean(name='test_loss')
test_acc = tf.metrics.Mean(name='test_accuracy')
def test_step(support, query, support_labels, query_labels, K, C, y):
    loss, mc_predictions, mean_accuracy, classwise_mean_acc, y = ProtoModel(support, query, support_labels, query_labels, K, C, y,n_times
    return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y
```

```
std tf.Tensor(0.60736364, shape=(), dtype=float32)
```

```
for epoch in range(1): #1000
    test_loss.reset_states()
    test_acc.reset_states()
    tquery_patches1, tsupport_patches1, query_labels1, support_labels1, x1 = test_episode(test_patches_class,test_class_labels,3,5,0,3)
    loss1, mc_predictions1, mean_accuracy1, classwise_mean_acc1, y1 = test_step(tsupport_patches1, tquery_patches1,support_labels1, query
    print('OA1',mean_accuracy1)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+1,classwise_mean_acc1[i])
    print('loss',loss1)
    tquery_patches2, tsupport_patches2, query_labels2, support_labels2, x2 = test_episode(test_patches_class,test_class_labels,3,5,3,6)
    loss2, mc_predictions2, mean_accuracy2, classwise_mean_acc2, y2 = test_step(tsupport_patches2, tquery_patches2,support_labels2, query
    print('OA2',mean_accuracy2)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+4,classwise_mean_acc2[i])
    print('loss',loss2)
```

```
std tf.Tensor(15.954748, shape=(), dtype=float32)
OA1 tf.Tensor(0.875, shape=(), dtype=float32)
class 1 [0.8780487804878049]
class 2 [0.8620689655172413]
class 3 [1.0]
loss tf.Tensor(93.63, shape=(), dtype=float32)
std tf.Tensor(13.083939, shape=(), dtype=float32)
OA2 tf.Tensor(0.9009901, shape=(), dtype=float32)
class 4 [1.0]
class 5 [0.85]
class 6 [1.0]
loss tf.Tensor(74.50175, shape=(), dtype=float32)
```

```
std tf.Tensor(0.6954355, shape=(), dtype=float32)
```

**Overall Accuracy**

```
std tf.Tensor(0.4709383, shape=(), dtype=float32)
```

```
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
overall_predictions = tf.concat([mean_predictions1,mean_predictions2],axis=0)
overall_true_labels = tf.concat([y1,y2],axis=0)
correct_pred = tf.cast(tf.equal(                                          # accuracy for the current pass
            tf.cast(tf.argmax(overall_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(overall_true_labels,axis=-1), tf.int32)), tf.float32)
o_acc = tf.reduce_mean(correct_pred)
print("Overall accuracy:",o_acc)
```

```
Overall accuracy: tf.Tensor(0.88814694, shape=(), dtype=float32)
```

```
std tf.Tensor(0.67121047, shape=(), dtype=float32)
```

### Confusion Matrix

```
std tf.Tensor(0.00959022, shape=(), dtype=float32)
```

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
cm_pred1 = tf.argmax(mean_predictions1, axis=-1)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
cm_pred2 = tf.argmax(mean_predictions2, axis=-1) + 3
overall_predictions = tf.concat([cm_pred1,cm_pred2],axis=0)
cm_true1 = tf.argmax(y1,axis=-1)
cm_true2 = tf.argmax(y2,axis=-1) + 3
overall_true_labels = tf.concat([cm_true1,cm_true2],axis=0)
results = confusion_matrix(overall_true_labels,overall_predictions)
print ('Confusion Matrix :')
print(results)
print ('Report : ')
print (classification_report(overall_true_labels, overall_predictions))
```

```
Confusion Matrix :
[[ 36   0   5   0   0   0]
 [ 32 200   0   0   0   0]
 [  0   0  23   0   0   0]
 [  0   0   0  15   0   0]
 [  0   0   0  11 170  19]
 [  0   0   0   0   0  88]]
Report :
              precision    recall  f1-score   support

           0       0.53      0.88      0.66        41
           1       1.00      0.86      0.93       232
           2       0.82      1.00      0.90        23
           3       0.58      1.00      0.73        15
           4       1.00      0.85      0.92       200
           5       0.82      1.00      0.90        88

    accuracy                           0.89       599
   macro avg       0.79      0.93      0.84       599
weighted avg       0.92      0.89      0.90       599

std tf.Tensor(0.2690313, shape=(), dtype=float32)
```

### Kappa Accuracy

```
std tf.Tensor(0.45349717, shape=(), dtype=float32)
```

```
import sklearn
sklearn.metrics.cohen_kappa_score(overall_true_labels, overall_predictions)
```

```
0.8486478128547347
std tf.Tensor(0.17669758, shape=(), dtype=float32)
std tf.Tensor(0.17989632, shape=(), dtype=float32)
std tf.Tensor(0.10158191, shape=(), dtype=float32)
std tf.Tensor(0.86817473, shape=(), dtype=float32)
std tf.Tensor(0.47178328, shape=(), dtype=float32)
std tf.Tensor(0.047749706, shape=(), dtype=float32)
std tf.Tensor(0.16953886, shape=(), dtype=float32)
std tf.Tensor(0.073957235, shape=(), dtype=float32)
std tf.Tensor(0.0775002, shape=(), dtype=float32)
std tf.Tensor(0.37206465, shape=(), dtype=float32)
std tf.Tensor(0.27704376, shape=(), dtype=float32)
std tf.Tensor(0.1227358, shape=(), dtype=float32)
std tf.Tensor(0.008589668, shape=(), dtype=float32)
std tf.Tensor(0.16826668, shape=(), dtype=float32)
std tf.Tensor(0.38758543, shape=(), dtype=float32)
std tf.Tensor(0.030121319, shape=(), dtype=float32)
std tf.Tensor(0.27752325, shape=(), dtype=float32)
std tf.Tensor(0.40059537, shape=(), dtype=float32)
std tf.Tensor(0.6221316, shape=(), dtype=float32)
std tf.Tensor(0.4246662, shape=(), dtype=float32)
std tf.Tensor(0.06968818, shape=(), dtype=float32)
std tf.Tensor(0.15054366, shape=(), dtype=float32)
std tf.Tensor(0.20421925, shape=(), dtype=float32)
std tf.Tensor(0.42376816, shape=(), dtype=float32)
std tf.Tensor(0.4112169, shape=(), dtype=float32)
std tf.Tensor(0.30860922, shape=(), dtype=float32)
std tf.Tensor(0.7089964, shape=(), dtype=float32)
std tf.Tensor(0.20682113, shape=(), dtype=float32)
std tf.Tensor(0.3676384, shape=(), dtype=float32)
std tf.Tensor(0.3337832, shape=(), dtype=float32)
std tf.Tensor(0.51341957, shape=(), dtype=float32)
std tf.Tensor(0.102263466, shape=(), dtype=float32)
std tf.Tensor(0.15269838, shape=(), dtype=float32)
std tf.Tensor(0.20046005, shape=(), dtype=float32)
std tf.Tensor(0.20919229, shape=(), dtype=float32)
std tf.Tensor(0.056562737, shape=(), dtype=float32)
std tf.Tensor(0.7952241, shape=(), dtype=float32)
std tf.Tensor(0.06826053, shape=(), dtype=float32)
```